

Biologically informed graph neural networks for perturba- tion effect prediction

– Integrating Multi-Modal Biological Data

Liuxi Mei

Supervisor : Oleg Sysoev
Examiner : Sebastian Mair

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

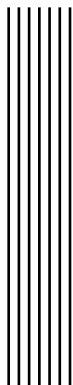
Abstract

Drug development is a long, expensive, and high risk process, and many candidate drugs fail before reaching the market. One reason is that drug effects are difficult to predict. A drug may bind to its target proteins, but the final cellular response depends on many interacting genes, proteins, and signaling pathways. Because these molecular systems are highly connected, it is difficult to predict how a drug perturbation will change gene expression in a specific cell type.

At the same time, large biological databases now provide useful information that can support this task. Some databases describe molecular relationships, such as gene–gene, protein–protein, and signaling interactions. Others provide information about drug targets and the genes affected by drug treatment. These resources make it possible to combine experimental perturbation data with prior biological knowledge.

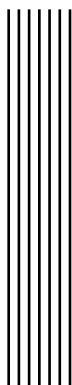
This work proposes a graph based model that combines a large molecular interaction network with drug target information to predict drug induced gene expression changes. The model uses the interaction network to represent how drug effects may spread through the cell, and it uses known drug targets to indicate where the perturbation starts. Such predictions can support drug development by helping researchers identify possible mechanisms of action, prioritize compounds for follow up experiments, and reduce the cost of experimental screening.

The proposed model is evaluated under unseen drug and unseen cell settings and is compared with baseline models, including a multilayer perceptron that does not use the gene interaction graph. Under a single dose and single time point setting, the graph model improves unseen cell performance by 24.4% relative to the multilayer perceptron baseline. The results show that combining prior molecular knowledge with drug target information can improve prediction of perturbation responses, and the learned attention weights provide an interpretable view of which interactions are emphasized by the model.



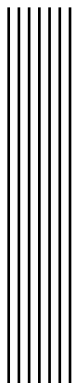
Acknowledgments

I would like to express my sincere gratitude to my supervisor, colleagues, and family for their continuous support and encouragement throughout the course of this research.



Contents

Abstract	iii
Acknowledgments	iv
Contents	v
List of Figures	vi
1 Introduction	1
1.1 Background	1
1.2 Drug perturbation	2
1.3 High-throughput experiment	2
1.4 Prior knowledge of molecular interactions	3
1.5 Aim	3
1.6 Related works	4
1.7 Ethical considerations	5
1.8 Limitations	6
2 Data	7
2.1 CMap LINCS Dataset	7
2.2 OmniPath Dataset	10
2.3 Data preprocessing	12
3 Method	16
3.1 Deep neural networks	16
3.2 Graph neural networks	18
3.3 Baseline model	22
3.4 Proposed model	23
3.5 Training	25
3.6 Evaluation	26
Bibliography	27



List of Figures

1.1	Conceptual illustration of the AstraZeneca “compound 12” case. Although the compound targets KRAS, additional off-target effects may spread through molecular networks and cause unwanted outcomes.	2
1.2	Sketch of the high-throughput well plate experiment, adapted from the LINCS L1000 assay description [1]. A well is a small, separate container that holds cells for individual testing.	3
1.3	Left: Example of a real gene network, showing complex connections. Right: A simple drawing of a graph neural network (GNN), where genes and proteins are nodes, and their interactions are edges.	4
2.1	LINCS L1000 data processing workflow. The pipeline converts raw fluorescence intensities (Level 1) to deconvoluted probe values (Level 2), normalized and inferred gene expression (Level 3), differential expression signatures (Level 4), and consensus signatures (Level 5).	8
2.2	Overview of the main data tables and how identifiers connect them. Expression profiles are linked to metadata by <code>distil_id</code> and to compound annotations by <code>pert_id</code> . The gene interaction graph is aligned to the same gene identifiers as the expression matrix. . .	10
2.3	Schematic overview of the OmniPath interaction tables and the graph construction used in this thesis. Each row in OmniPath describes a directed regulator to target relationship together with sign related fields (stimulation or inhibition) and consensus annotations. We filter to retain directed regulatory relationships with strong consensus and then convert the resulting edge list into a signed gene graph for downstream message passing. . .	11
2.4	Effect of successive filtering steps on the number of treatment profiles retained for modelling in a single dose and single time point setting. The final step counts treatment profiles that can be paired with at least one matched vehicle control under the context matching rules.	12
2.5	Distribution of treatment response vectors $x_{\text{deg}} = x_{\text{trt}} - x_{\text{ctl}}$ computed from matched treatment and control pairs in the 978 landmark gene space. Most responses are subtle with near zero mean shifts, while a small subset of genes shows large positive or negative changes.	13
2.6	Visualization of the OmniPath derived interaction graph in the 978 landmark gene setting. Node color indicates node degree.	15
3.1	A fully connected neural network. An input vector is mapped to an output vector by stacked hidden layers.	18

3.2	Main components of a graph. A graph can contain node attributes, edge attributes, and global information.	18
3.3	Illustration of message passing and node update in a GNN. Messages from neighbouring nodes are aggregated and then used to update the target node representation.	19
3.4	Illustration of multi layer GCN propagation. The input graph is processed by consecutive graph convolution layers, and the learned representations can be used for node level or graph level prediction.	21
3.5	Illustration of the attention mechanism in GAT. Node features are linearly projected, concatenated, and transformed into attention weights through LeakyReLU and softmax. . . .	22
3.6	Structure of the baseline model. Drug target information, cell identity, and control gene expression are used as input to a feed forward network that predicts the gene expression response.	23
3.7	Structure of the proposed model. The model combines the biological interaction graph, control expression profile, drug target information, and cell identity to produce gene wise predictions through graph attention layers and node level prediction heads.	24



1 Introduction

1.1 Background

Drug discovery is a long, expensive, and uncertain process. Many candidate drugs fail before they reach the market, even after spending a great deal of time and money [2]. One reason is that a drug does not act in isolation. It may bind to one target protein, but the final response depends on many interacting genes, proteins, and signaling pathways inside the cell [3, 4]. Because these molecular systems are highly connected, it is difficult to predict how a drug will change the state of a cell. Better prediction of cellular responses can therefore help researchers screen compounds earlier and focus experiments on more promising candidates.

This difficulty is illustrated by the AstraZeneca “compound 12” case reported in 2025. The compound was designed to act on KRAS, but later analysis showed that it also affected unintended targets and led to harmful toxic effects in animal studies [5]. The case illustrates an important point. A drug may engage its intended target and still produce an unwanted outcome because its effects spread through a larger molecular system. This thesis focuses on that broader prediction problem. The goal is to predict how gene expression changes after drug treatment, compared with a matched control, in a specific cell type.

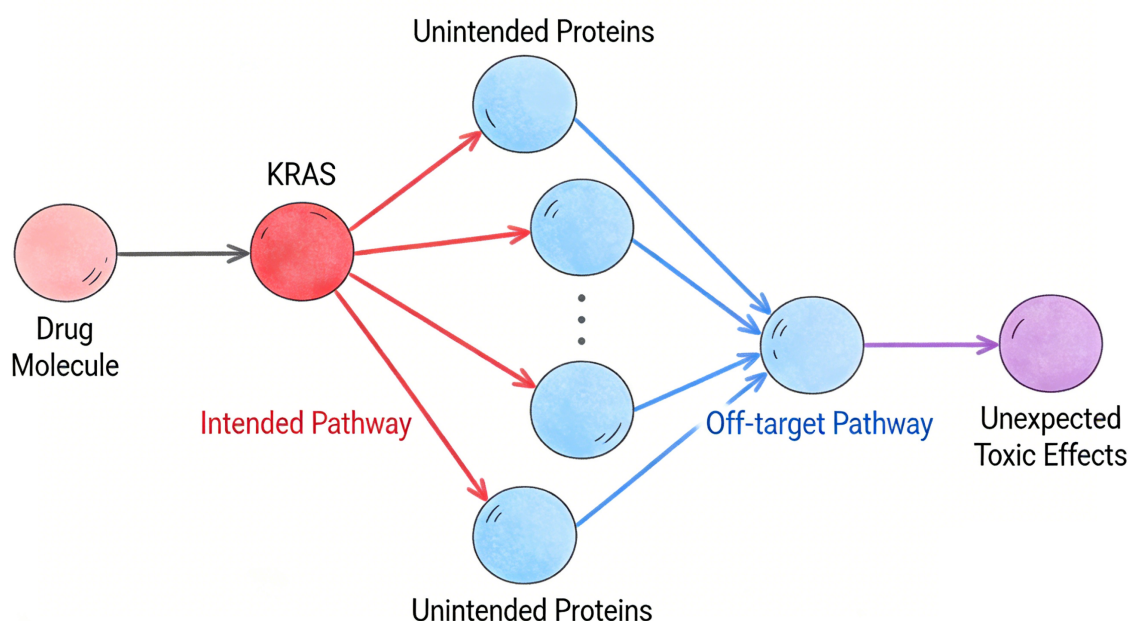


Figure 1.1: Conceptual illustration of the AstraZeneca “compound 12” case. Although the compound targets KRAS, additional off-target effects may spread through molecular networks and cause unwanted outcomes.

1.2 Drug perturbation

Drug perturbation refers to the use of a drug to change the state of a cell so that the cellular response can be studied. A drug usually starts by binding to one or more target proteins. This initial event may block a protein, activate it, or otherwise change its function. However, the effect usually does not stop at the first target. The signal can spread through molecular interactions inside the cell and eventually influence many downstream genes.

In this study, the cellular response is described through gene expression. Gene expression measures how active each gene is. When a drug perturbs a cell, some genes become more active and others become less active. Predicting these changes is useful because they summarize the overall molecular response of the cell in a form that can be measured systematically.

1.3 High-throughput experiment

To study drug perturbations at large scale, researchers use high-throughput experiments. The main idea is simple. Many cell lines are grown in multi-well plates, many drugs are applied under controlled conditions, and gene expression is measured after treatment. This design makes it possible to compare many perturbations in a standardized way.

In this thesis, we use the LINCS L1000 dataset. LINCS records gene expression responses for many compounds, cell lines, doses, and treatment times. Each well contains cells under one experimental condition. After a fixed treatment period, gene expression is measured for 978 landmark genes, and the expression of additional genes is inferred computationally from these measurements.

Each plate includes two main types of wells:

1. **Treatment wells:** These contain cells exposed to a specific drug, usually dissolved in DMSO.
2. **Control wells:** These contain cells exposed only to DMSO [6], without the active drug. They provide a reference for the same experimental setting without the drug effect.

The core measurement is the difference between a treatment well and a matched control well. This comparison helps isolate the effect of the drug from the background state of the cell. In practice, a control is useful only when it is matched to a comparable treatment condition. The dataset therefore also records the cell line, drug dose, treatment time, and plate information. These variables are important because the same drug can act differently in different cell types and under different experimental settings.



Figure 1.2: Sketch of the high-throughput well plate experiment, adapted from the LINCS L1000 assay description [1]. A well is a small, separate container that holds cells for individual testing.

1.4 Prior knowledge of molecular interactions

A second source of information comes from biological knowledge databases. These resources summarize known relationships among genes and proteins, such as which molecules regulate others or pass signals downstream. Such information is useful because drug effects do not arise from one target alone. A drug may start from a small set of targets, but the response is shaped by how signals move through the surrounding molecular network.

OmniPath is one example of such a database. It integrates molecular interaction information from many curated resources and provides directed relationships that can be used to build a graph representation of the cell [7]. In this thesis, the OmniPath network is aligned with the genes measured in LINCS L1000 so that prior knowledge and experimental data are represented in the same gene space. This combination is central to the thesis. Experimental perturbation data show what changes after treatment, while a molecular interaction network provides information about how those changes may spread through the cell. Drug target information adds the starting points of the perturbation. By combining these sources, the model can make predictions that are more informed than predictions based on expression data alone.

1.5 Aim

This thesis studies how to predict gene expression changes after drug perturbation. The input includes the baseline state of the cell, known drug targets, and a molecular interaction network. The motivation is practical as well as methodological. If such responses can be predicted more accurately, researchers can use the predictions to screen compounds more efficiently, prioritize follow up experiments, and better understand possible drug mechanisms.

The main idea is to combine two forms of prior information. One is a molecular interaction network from databases such as OmniPath. The other is drug target information that indicates where the perturbation starts. This thesis investigates whether a graph based model can combine these sources of information to improve prediction of perturbation responses.

The aims of this thesis are:

1. To build a graph neural network (GNN) based on the OmniPath molecular interaction network. This model will predict drug-induced gene expression changes by combining baseline cell data and drug target information.
2. To test how well the model works on unseen drugs and unseen cell lines (where test drugs or cells were never seen during training).
3. To compare this model with baseline models that do not use a gene network, and to run tests to see how important the drug target information is.

In our model, the graph represents molecular interactions, and the nodes correspond to the genes measured in the experiment. A graph attention network (GAT) [8] is used because it can learn different weights for different neighbours. This is useful when the interaction graph is incomplete or noisy. The learned attention weights can also be inspected as an interpretation signal.

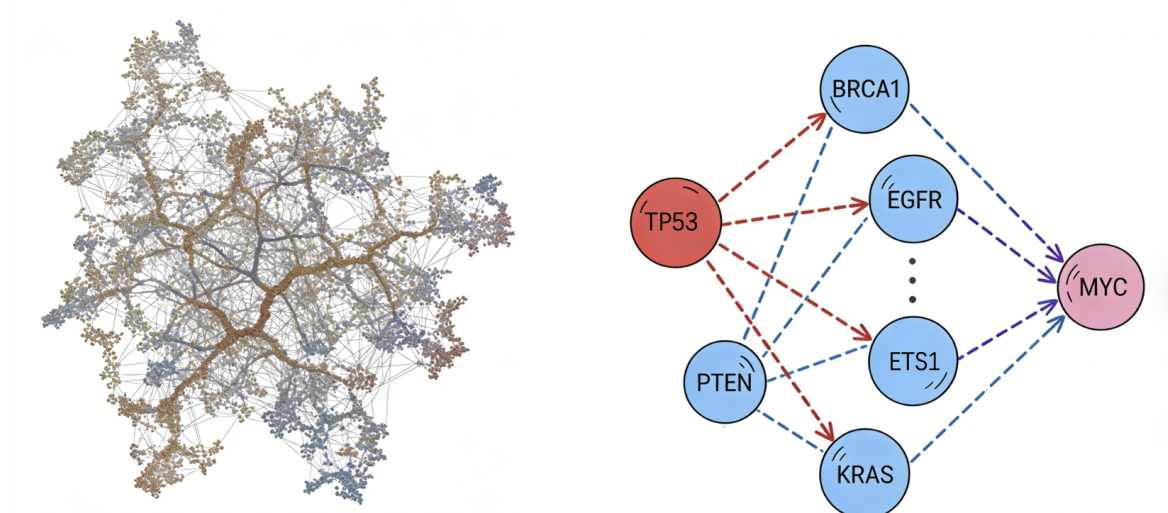


Figure 1.3: Left: Example of a real gene network, showing complex connections. Right: A simple drawing of a graph neural network (GNN), where genes and proteins are nodes, and their interactions are edges.

Figure 1.3 gives a conceptual view of this modelling idea. The molecular network provides the graph structure, and the GNN uses this structure to propagate information from drug targets to downstream genes. The model is evaluated not only on random splits, but also on unseen drugs and unseen cell lines, so that generalization is tested under more realistic conditions.

1.6 Related works

Graph neural networks (GNNs) are now widely used in biology because many biological systems are naturally represented as graphs. A recent review summarizes GNN applications in single-cell omics and discusses common architectures such as GCNs, GraphSAGE, and GATs [9, 10, 11, 12, 8]. These models are attractive because they allow each node to update its representation by using both

its own features and information from connected neighbours. In biological settings, this makes them a natural choice for modelling pathways, regulatory interactions, and other structured molecular systems.

Several studies have already shown the value of graph-based learning in perturbation biology. GEARS uses a gene relationship graph to predict responses to genetic perturbations in single-cell data [13]. Cellograph models multi-condition single-cell datasets with graph-based learning [14]. Evans et al. also explore graph-structured neural networks for perturbation biology more broadly [15]. Although these studies focus on different data types and tasks, they support the general idea that graph structure can help connect an intervention to downstream molecular changes.

A different line of work predicts drug effects without explicitly using a biological interaction graph. Large resources such as LINCS L1000 and Connectivity Map make this possible because they provide gene expression responses for many compounds and cell lines [1]. DeepCOP [16] is an example of this direction. It uses a multilayer perceptron and drug information to predict gene regulation effects. This type of model is simple and scalable, and it provides a strong baseline. However, it treats genes as independent outputs and does not use known molecular connections among genes and proteins. Another direction uses prior biological knowledge more explicitly. Regulation Flow Analysis models how perturbation signals may spread through a curated interaction network and uses this structure to infer possible mechanisms [17]. This kind of method is attractive because it is biologically interpretable. Its limitation is that the propagation rule is largely fixed by the network and does not learn directly from large perturbation datasets. In contrast, data-driven graph neural networks can learn how strongly information should propagate while still respecting the network structure.

Some recent studies combine graph structure with supervised prediction for drug response related tasks [18, 19]. These studies suggest that molecular networks can provide a useful inductive bias when labels are noisy or when the response is produced through many indirect molecular steps. They also show that the quality of the graph matters. Curated resources such as OmniPath [7] are therefore attractive because they provide directed interactions and sign information, which are useful for modelling signal propagation.

Evaluation is another important issue in this literature. Many studies report performance under random splits, where related drugs or cellular contexts may appear in both training and test sets. For perturbation prediction, this can overestimate how well a model generalizes. A more demanding setting is to test unseen drugs or unseen cell lines. This thesis follows that perspective and studies whether a graph-based model can generalize beyond the conditions observed during training.

1.7 Ethical considerations

This thesis uses public data from LINCS L1000 and OmniPath. The data do not contain personal patient identifiers, and the work does not involve human or animal subjects. The main ethical concern is the responsible use of model outputs.

The model predicts gene expression changes after drug treatment. These predictions should be treated as computational hypotheses rather than evidence of efficacy or safety. They may support compound prioritization and hypothesis generation, but they should not be used for medical decision making without experimental and clinical validation.

Responsible reporting is therefore important. Model performance should be reported together with failure cases and limits of interpretation, especially when predictions are applied to unseen drugs or cell lines.

Computational cost is another consideration. Training deep learning models requires compute resources, and repeated hyperparameter search increases energy use. In this thesis, the models are trained on standard hardware with a limited set of configurations to keep resource use at a reasonable level.

1.8 Limitations

This study inherits limitations from both the experimental data and the prior knowledge source. LINCS profiles are measured in a limited set of cell lines, many of which are cancer derived. The results may therefore not transfer directly to primary cells or tissues. Experimental factors such as dose, treatment time, and plate context can also affect the measurements. Matched controls reduce some shared technical variation, but residual bias may remain.

OmniPath is built from published studies, and the available knowledge is uneven across pathways and genes. Well studied mechanisms tend to have more recorded interactions, while less studied mechanisms may be under represented. This imbalance can affect both prediction quality and interpretation.

The model also provides attention weights as an interpretation signal. These values indicate which edges receive stronger weight during message passing, but they do not establish biological causality. A highlighted path may be consistent with known biology, and it may also reflect graph structure, data bias, or measurement noise.

Reproducibility is also an important practical limitation. Because model training includes random elements, clear reporting of data splits, settings, and key hyperparameters is necessary for fair comparison and reliable interpretation.



2 Data

This chapter describes the main data sources and data structures used in this thesis. Gene expression profiles from the CMAP LINCS 2020 dataset and biological network information from the OmniPath database provide two complementary views of the problem. The expression data describe how cells respond to treatments, while the network data describe how genes and proteins interact. Together, they support models that combine experimental measurements with prior biological knowledge.

The data used in this thesis can be organized into three linked parts. The first part is the gene expression matrix, which stores treatment and control profiles. The second part is the metadata and compound annotation tables, which record the cell line, dose, time, plate information, and drug target annotations for each experiment. The third part is the OmniPath interaction network, which provides prior knowledge about relationships among genes and proteins. The following sections explain how these parts are structured and how they are linked through shared identifiers.

2.1 CMap LINCS Dataset

The NIH Common Fund’s Library of Integrated Network based Cellular Signatures (LINCS) provides large scale perturbation experiments for studying how cells respond to chemical and genetic interventions. A major component of LINCS is the L1000 assay, which directly measures the expression of 978 carefully selected landmark genes [1]. These landmark genes were chosen because they capture most of the variation in gene expression across many biological conditions. To expand gene coverage, the L1000 pipeline uses a statistical model to infer the expression of additional genes based on the measured landmark genes, resulting in a final gene space of 12,328 genes: 978 directly measured and 11,350 inferred. The L1000 data are widely used because they cover many perturbagens, cell lines, and experimental settings, together with matched control samples.

L1000 Data Processing Flow

LINCS L1000 data have been released through both the GEO archive (Phase I and Phase II series) and later consolidated resources such as the CMap/LINCS 2020 release. Across releases, L1000 data are commonly organized into several *data levels*, which represent different representations of the same experiments:

1. **Level 1: Raw fluorescence intensities.** These are the direct measurements from the L1000 assay, reflecting the signal for each probe before any normalization or correction.

2. **Level 2: Deconvoluted probe values.** Probe signals are mapped to gene level values, resolving cases where multiple probes target the same gene.
3. **Level 3: Normalized and inferred gene expression.** Data are normalized across samples and batches, and the expression of non landmark genes is computationally inferred using a regression model trained on landmark genes. This produces a consistent gene expression matrix for downstream analysis.
4. **Level 4: Differential expression signatures.** Treatment and control profiles are compared to compute differential expression (for example, z scores), highlighting the effect of perturbations.
5. **Level 5: Consensus signatures.** Multiple replicates and conditions are aggregated to produce robust consensus signatures for each perturbation.

Level 3 data are used in this study, which provide normalized and inferred gene expression profiles in a unified gene space across samples.

L1000 Data Processing Workflow: Levels 1–5

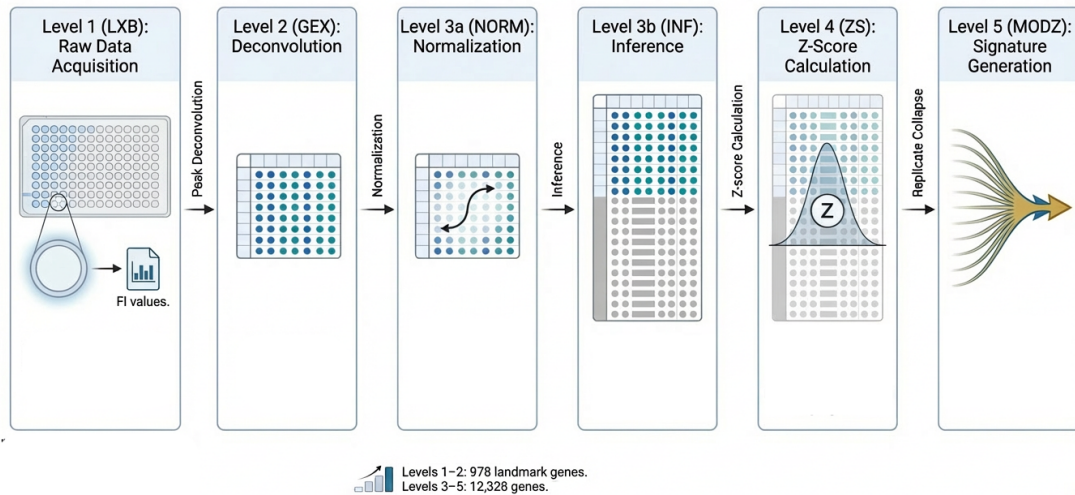


Figure 2.1: LINCS L1000 data processing workflow. The pipeline converts raw fluorescence intensities (Level 1) to deconvoluted probe values (Level 2), normalized and inferred gene expression (Level 3), differential expression signatures (Level 4), and consensus signatures (Level 5).

Each experiment at Level 3 is represented as a vector of gene expression values in the shared gene space. We denote a profile as $x \in \mathbb{R}^G$, where $G = 12,328$ for the full gene set, and $G = 978$ when only the landmark genes are used. Profiles are indexed by unique identifiers (for example, `distil_id`), and they are stored in large matrices where rows correspond to profiles and columns correspond to genes in a fixed order. A separate gene information table provides the mapping between column indices and gene identifiers.

Treatment and control data

LINCS experiments include both compound treatment profiles and matched vehicle control profiles, covering diverse cell lines, compounds, and experimental conditions. Each profile is represented as a vector of gene expression values in the shared gene space. The associated experimental metadata (for example, cell line, compound, dose, and time) are stored in a separate table and linked to profiles through the identifier `distil_id`. In practical terms, treatment profiles describe the cellular state after drug exposure, while control profiles describe the same experimental setting without the active

drug. The comparison between these two profile types is the basis for the perturbation response used later in the thesis.

Metadata

Metadata are taken from the LINCS signature annotation table, which is distributed with the Level 3 expression data release [1]. Each row in this table describes a perturbation signature, including the compound identity, cell line, experimental conditions, quality control, and batch related variables. Metadata is critical for providing biological and experimental context to the raw expression matrices. It also provides the keys that link expression profiles to compound annotations. Without metadata, it would not be possible to determine which drug was used, which cell line was tested, which samples should be compared as treatment and control, or whether a profile passed quality control. As a result, reliable pairing, grouping, filtering, and model **evaluation would not be possible.**

Identifiers. Each signature is associated with a compound identifier `pert_id` and a cell line name `cell_line`. Each signature also has a unique signature identifier `sig_id`. The field `distil_ids` contains one or more expression profile identifiers (`distil_id`) that reference the corresponding rows in the Level 3 expression matrices.

Experimental condition variables. The metadata record the perturbation dose and time, including `pert_dose`, `pert_dose_unit`, `pert_time`, and `pert_time_unit`. These variables describe the experimental settings under which each signature was measured.

Batch and plate context. Measurements can be affected by small differences in the experiment. The metadata therefore include a bead batch identifier (`bead_batch`) and detection plate information (`det_plates`, `det_wells`). The bead batch tells us which batch of assay beads was used. These variables are useful because two samples may look different for technical reasons, even if their biology is similar. **Without this information, treatment and control samples may be matched under different experimental conditions, and part of the measured difference may come from the experiment itself rather than from the drug.**

Quality control. Quality control is used to check whether a measured profile is strong enough and consistent enough to be trusted in later analysis [1]. In LINCS, this is important because some signatures may be weak, noisy, or unstable across replicates. The metadata therefore include quality related fields such as `is_hiq` and `qc_pass`. Additional summary fields such as `tas` and related recall metrics describe how strong and reproducible a signature is. Consequently, these variables facilitate the identification of profiles that are more reliable and the removal of profiles that may add noise to the dataset.

Drug target

Drug related information is collected from the compound annotation table. This table is indexed by `pert_id` and provides compound names (`cmap_name`), a target field (`target`), and mechanism of action text (`moa`). It serves as the main link between perturbation signatures in LINCS and annotated information about the compounds. In this thesis, the table is used to identify which genes are reported as targets of each drug, so that drug specific input signals can be aligned with the gene space used by the expression matrix and the OmniPath graph.

Target annotations The `target` field lists reported protein or gene targets for a subset of compounds. A single drug may have multiple targets, so the `target` field can contain a list of genes rather than a single entry. In the preprocessing pipeline, these target names are mapped to the gene identifiers used in the selected expression space. This mapping step is necessary because only targets that can be aligned with the expression matrix and the graph can be used by the model. Importantly, target information is not available for all compounds in LINCS. Only a subset of perturbagens have

target annotations. This is expected because many compounds are not fully characterized. Their molecular targets may be incomplete or only partly known, and some compounds are identified from their observed effects in cells rather than from known targets.

Table association

This subsection summarizes how the main tables are linked through primary identifiers. The purpose of this linkage is to make each expression profile traceable to its experimental context (cell line, dose and time, batch/plate) and to its compound annotation. This enables reproducible grouping analyses and supports the construction of matched treatment and control comparisons in the pre-processing pipeline. Figure 2.2 gives an overview of these connections and shows how expression profiles, metadata, drug targets, and the interaction graph are aligned.

Profile key Level 3 expression matrices are indexed by `distil_id`. The metadata table contains a field `distil_ids`, which stores one or more `distil_id` values for each signature. This provides the link from metadata to expression profiles.

Compound key Each signature row in the metadata table includes a compound identifier `pert_id`. The compound annotation table uses the same `pert_id` as its primary key. This provides the join from signatures to drug target annotations.

Context variables for grouping. The metadata also include cell context (`cell_iname`) and batch/plate variables (for example, `bead_batch` and `det_plates`). These fields support grouping and stratified analysis by biological and technical context.

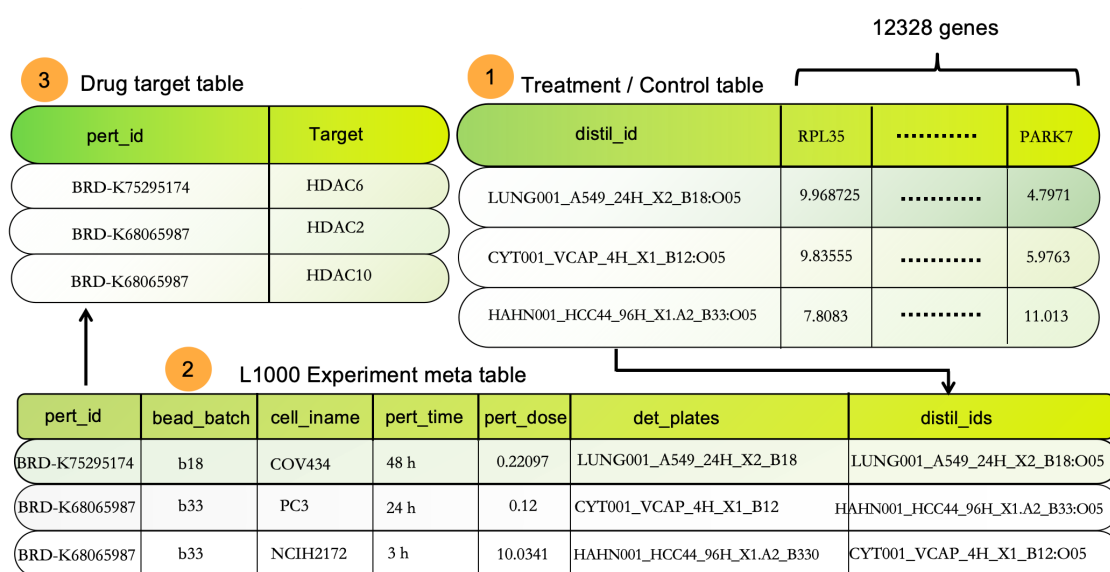


Figure 2.2: Overview of the main data tables and how identifiers connect them. Expression profiles are linked to metadata by `distil_id` and to compound annotations by `pert_id`. The gene interaction graph is aligned to the same gene identifiers as the expression matrix.

2.2 OmniPath Dataset

OmniPath is a public database that collects information about molecular interactions, signaling pathways, and regulatory relationships in human cells. It combines data from many pathway and transcription factor resources and links them into a single interaction table. In this thesis, we use the

transcription factor regulon part of OmniPath, where each row describes a regulator, a target gene, and whether the interaction is activating or inhibiting.

For each interaction, OmniPath stores several fields that describe the direction and sign of the relationship, together with evidence related metadata. For example, the columns `is_directed`, `is_stimulation` and `is_inhibition` indicate whether the interaction is directed and whether it is activating or inhibiting. OmniPath also records evidence strength through fields such as `sources`, `n_sources`, `n_primary_sources` and `n_references`. The column `references` lists supporting entries together with publication identifiers, and `references_stripped` provides cleaned identifiers.

OmniPath integrates interactions from many resources, so different sources may provide conflicting annotations for the same interaction, for example reporting stimulation in one database and inhibition in another. OmniPath records both the individual annotations and consensus fields, which can be used to identify and handle such conflicts during graph construction.

OmniPath also provides evidence counts for each edge, such as `n_sources`, `n_primary_sources`, and `n_references`. These fields summarize how many databases and publications support an interaction. When sources disagree, this can be noise, but it can also reflect regulation that depends on cell type or experimental conditions. In our graph construction, we use consensus and evidence counts to reduce uncertainty, while accepting that the final network may still contain imperfect edges. In our setting, genes from the LINCS expression matrix are mapped to nodes in the OmniPath regulon graph. The resulting directed network encodes transcription factor gene relationships that are supported by published studies. When we combine this network with the gene expression profiles, the model can use both experimental measurements and prior regulatory information, which helps interpret predictions in a biological context. Figures 2.3 and 2.6 illustrate both the table structure used to build the graph and the final graph after filtering and alignment.

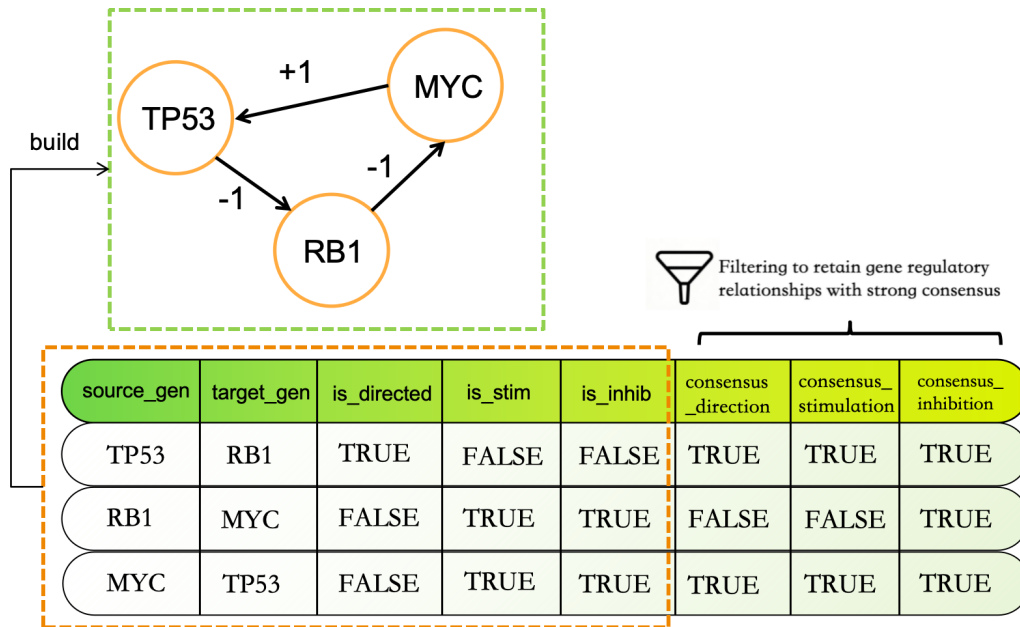


Figure 2.3: Schematic overview of the OmniPath interaction tables and the graph construction used in this thesis. Each row in OmniPath describes a directed regulator to target relationship together with sign related fields (stimulation or inhibition) and consensus annotations. We filter to retain directed regulatory relationships with strong consensus and then convert the resulting edge list into a signed gene graph for downstream message passing.

2.3 Data preprocessing

This section describes how the raw LINCS Level 3 matrices and metadata are converted into the aligned tensors used by the models. The implementation follows the data loading pipeline used in our code. The goal of preprocessing is:

1. select a consistent gene set;
2. build matched treatment and control pairs;
3. compute treatment response targets;
4. attach drug features that can be used as model inputs.

CMap LINCS L1000 preprocessing

Gene set selection. Two gene spaces are supported. In landmark mode, the 978 landmark genes are used and the `Entrez gene identifiers` are kept in a fixed order defined by the landmark gene list file. In full gene mode, all 12,328 genes are loaded from the LINCS gene information table and the `pr_gene_id` column is used as the identifier. In both cases, a single fixed gene order is enforced across all `tensors` (control profiles, treatment profiles, response targets, and drug vectors). This consistent ordering makes downstream matrix operations well defined and ensures that model inputs and outputs align with the correct gene identifiers.

Metadata filtering. Preprocessing starts from the meta table, which links each profile to its cell line, compound identifier, perturbation dose, perturbation time, and batch information. Only two profile types are retained: compound treatments (`trt_cp`) and vehicle controls (`ctl_vehicle`). To reduce heterogeneity across experiments, a single time point and a single dose setting are used. By default, profiles measured at 24 hours and treatments with dose close to 10 μ M are kept. This choice follows a widely used LINCS condition and improves comparability by reducing variation caused by different exposure times. In addition, only high quality treatment profiles are retained (for example, `is_hiq` = 1). Optional filtering by cell line is supported by restricting entries in `siginfo_beta.txt`.

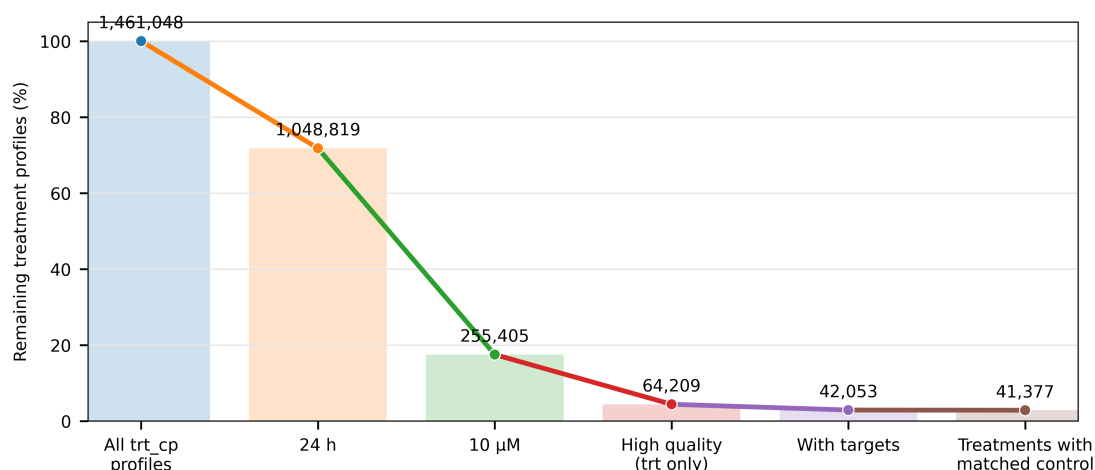


Figure 2.4: Effect of successive filtering steps on the number of treatment profiles retained for modelling in a single dose and single time point setting. The final step counts treatment profiles that can be paired with at least one matched vehicle control under the context matching rules.

Control profile. From the filtered metadata, control profiles are collected and the `distil_ids` field is expanded so that each profile has a unique identifier. For each control profile, the associated cell

line, bead batch, and detection plate are recorded. The expression vectors for these control identifiers are then loaded from the Level 3 matrix. The loader reads only the requested identifiers and only the selected gene columns, which reduces memory and keeps a consistent gene order.

Treatment profile. The same procedure is applied to treatment profiles. The field `distil_ids` is expanded for each treatment record, and the corresponding cell line, compound identifier (`pert_id`), bead batch, and detection plate are stored for each sample. The expression vectors for these treatment identifiers are then loaded from the Level 3 matrix.

Matched pairing and target. For each treatment profile, control candidates are selected from the same cell line and the same experimental context. The primary rule is to match cell line, bead batch, and detection plate. If no control exists for this exact match, a relaxed rule is used that matches cell line and detection plate. If no suitable control exists, the treatment profile is removed from the dataset. From the resulting candidate pool, a fixed number of controls is sampled at random for each treatment profile. In the main experiments, the default number is 3 controls per treatment. If the candidate pool contains fewer than 3 controls, sampling is performed with replacement. Each sampled treatment–control pair is then treated as one modelling example. This design increases the number of paired examples and reduces sensitivity to noise in any single control profile. The supervised target is the treatment response vector

$$y = x_{\text{trt}} - x_{\text{ctl}},$$

where x_{trt} is the treatment expression vector and x_{ctl} is the matched control expression vector. This construction removes much of the baseline cell state and focuses learning on the change in gene expression induced by drug perturbation. When controls are measured in a similar experimental context, the subtraction removes shared technical noise, such as batch or plate effects, and makes the target easier to predict.

Because one treatment profile can be paired with multiple sampled controls, repeated paired examples may share the same treatment identifier. In the implemented evaluation pipeline, data splitting is applied after pair construction. In the cold drug and cold cell settings, all repeated pairs inherit the same drug or cell label and therefore remain on the same side of the split. This reduces leakage from repeated treatment profiles across training and test sets.

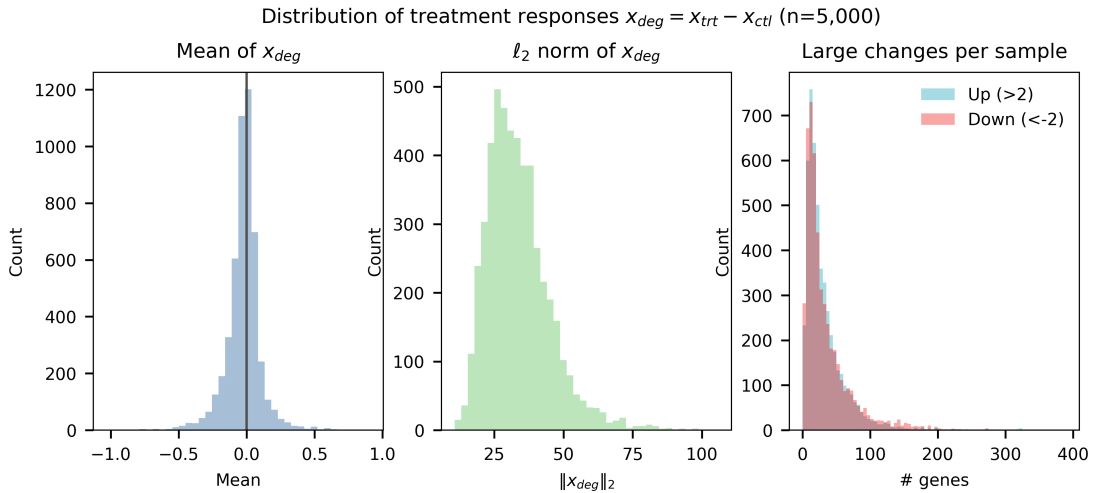


Figure 2.5: Distribution of treatment response vectors $x_{\text{deg}} = x_{\text{trt}} - x_{\text{ctl}}$ computed from matched treatment and control pairs in the 978 landmark gene space. Most responses are subtle with near zero mean shifts, while a small subset of genes shows large positive or negative changes.

Drug feature. Each training example is linked to a compound identifier. A sparse target annotation vector over the selected gene set is constructed by mapping annotated target gene symbols to the gene identifiers used in the expression matrix.

Example filtering. Only treatment examples with available drug target annotations are retained. Examples whose `pert_id` cannot be linked to an existing `target` field are removed.

Outputs. After preprocessing, the dataset contains matched arrays for control gene expression, drug target vectors, and response targets. Identifiers for cell line, bead batch, detection plate, and profile ids are also retained. These identifiers are used for evaluation, grouping analysis.

OmniPath graph preprocessing

In this thesis, the prior interaction graph is constructed by merging the transcription factor regulon table and the interaction table. This merge increases edge coverage, so more genes have neighbours that are useful in the expression data during message passing. Each row describes a source gene and a target gene together with sign information indicating stimulation or inhibition when available.

Identifier alignment. OmniPath tables are provided in gene symbols. These symbols are mapped to the gene identifiers used by the expression matrices. Edges are retained only when both endpoints can be mapped and both genes are present in the selected gene set. This ensures that graph nodes, expression vectors, and drug target vectors refer to the same ordered gene list.

Direction and sign. The graph is represented as a directed edge list. When OmniPath provides sign annotations, each edge is assigned a sign value $w \in \{+1, -1\}$ corresponding to stimulation or inhibition. If sign information is missing for an edge, it can be excluded or treated as unknown depending on the experiment setting.

Deduplication and outputs. An interaction may appear multiple times across sources. Duplicate edges are merged so that each ordered pair (u, v) appears once in the final graph. The resulting graph is stored as `edge_index` (a $2 \times E$ integer tensor) and optional edge attributes such as signed weights. This representation is used directly by the graph neural network for message passing.

Graph size. In the landmark gene setting used in this thesis, the resulting OmniPath graph contains 978 nodes and 1,627 directed edges after filtering and deduplication. The node set matches the 978 dimensional landmark expression space, so graph outputs align directly with the prediction targets. The graph is relatively sparse (average degree on the order of 1–2), which helps focus message passing on higher confidence interactions, reduces noise propagation, and keeps computation efficient. When the full gene space is used ($G = 12,328$), the constructed OmniPath graph contains 12,328 nodes and 26,166 directed edges under the same preprocessing settings.

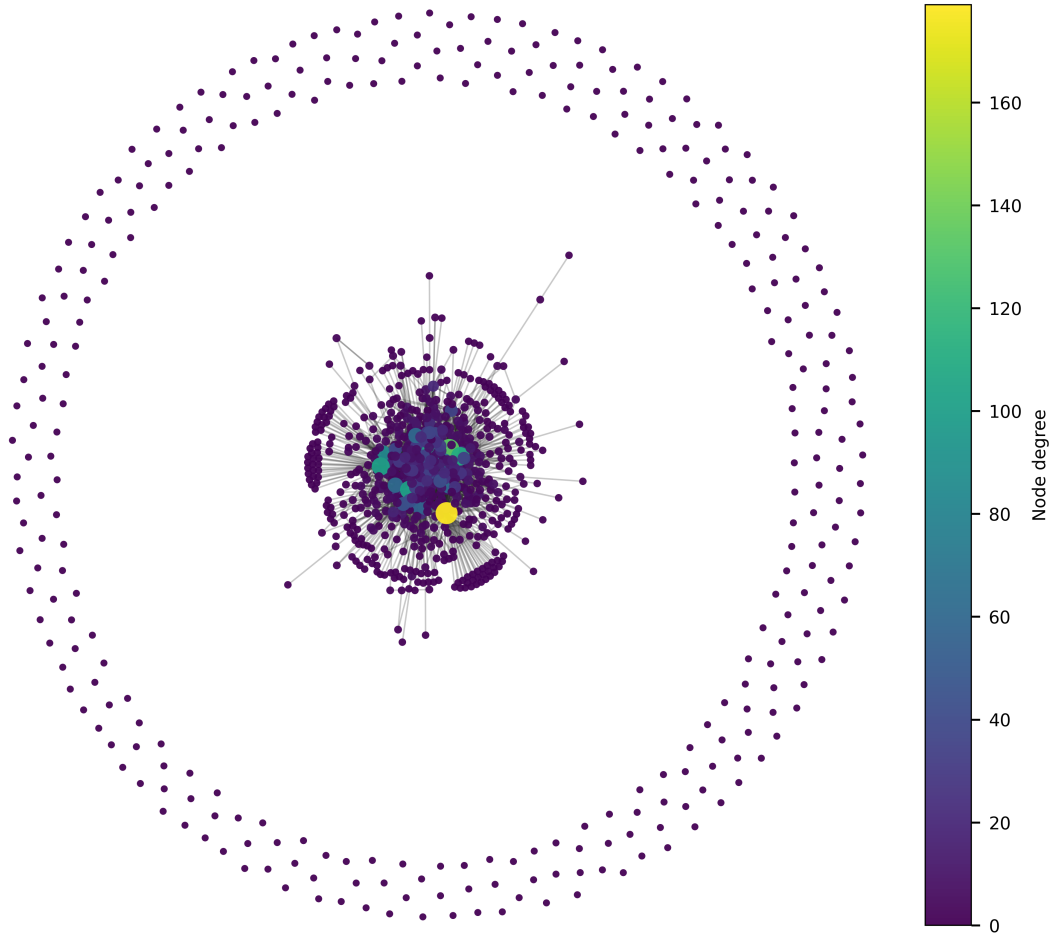


Figure 2.6: Visualization of the OmniPath derived interaction graph in the 978 landmark gene setting. Node color indicates node degree.

Summary

After preprocessing, each modelling example contains a matched control expression profile x_{ctl} , a treatment response target $y = x_{\text{trt}} - x_{\text{ctl}}$, a binary drug target vector aligned to the same gene order, and a cell identity label. These sample level inputs are combined with a fixed OmniPath derived interaction graph.

Two gene spaces are supported. In landmark mode, the data use 978 genes and the aligned OmniPath graph contains 978 nodes and 1,627 directed edges. In the full gene setting, the data use 12,328 genes and the aligned OmniPath graph contains 12,328 nodes and 26,166 directed edges. In both settings, a single gene order is used across control profiles, treatment profiles, response targets, drug target vectors, and graph nodes.

Only treatment examples that can be linked to matched controls and valid drug target annotations are retained for modelling. In practice, this means that the final dataset contains paired treatment control examples for which the response target and drug target information are both available.

3

Method

This thesis studies a prediction problem for drug perturbation experiments. Let G denote the number of genes in the selected gene set. For each sample, $x_{\text{ctl}} \in \mathbb{R}^G$ denotes the control expression profile and $x_{\text{trt}} \in \mathbb{R}^G$ denotes the corresponding treated profile. The supervised target is the response vector

$$y = x_{\text{trt}} - x_{\text{ctl}} \in \mathbb{R}^G. \quad (3.1)$$

The response target focuses on the change after treatment. It reduces shared measurement noise when the control and treatment are matched. It also makes targets more comparable across experiments. To make sure that the baseline model and the GNN model are comparable, all models in this thesis take the same type of inputs. The inputs include the control profile x_{ctl} and drug target related features such as target annotations. The model output is a predicted response vector $\hat{y} \in \mathbb{R}^G$. The learning goal is to make $\hat{y}$ close to y on new samples.

3.1 Deep neural networks

Layers

A deep neural network is built from layers. Each layer transforms its input to produce an output. The simplest layer is a dense layer. It applies a linear transformation followed by a nonlinear function. Given an input vector x of size d_{in} , a dense layer computes

$$h = \sigma(Wx + b), \quad (3.2)$$

where W is a weight matrix of size $d_{\text{out}} \times d_{\text{in}}$, b is a bias vector of size d_{out} , and σ is an activation function. The weight matrix determines how much each input feature contributes to each output. The bias shifts the output. Both are learned during training. The output size d_{out} is a choice. A larger output gives the layer more capacity to represent patterns. A smaller output forces the layer to compress information. The layer transforms the input space into a new space that is more useful for the task.

Activation Functions

The activation function σ adds nonlinearity. Without it, stacking layers would be equivalent to a single linear transformation. Common choices include:

$$\text{ReLU}(x) = \max(0, x) \quad (3.3)$$

ReLU outputs zero for negative inputs and the input value for positive inputs. It is simple and effective. It helps networks train faster by avoiding the vanishing gradient problem.

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (3.4)$$

Sigmoid squashes inputs into the range $(0, 1)$. It is smooth and differentiable. It is often used for binary classification outputs.

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (3.5)$$

Tanh squashes inputs into the range $(-1, 1)$. It is similar to sigmoid but centered around zero.

Stacking Dense Layers

Multiple layers are stacked to create a deep network. The output of one layer becomes the input to the next. This allows the network to learn hierarchical representations. Lower layers learn simple patterns. Higher layers combine these patterns into more complex ones. A network with L layers can be written as

$$\begin{aligned} h^{(0)} &= x \\ h^{(\ell+1)} &= \sigma(W^{(\ell)}h^{(\ell)} + b^{(\ell)}) \quad \text{for } \ell = 0, 1, \dots, L-1 \\ \hat{y} &= h^{(L)} \end{aligned} \quad (3.6)$$

where $h^{(0)}$ is the input, $h^{(\ell)}$ is the output of layer ℓ , and $\hat{y}$ is the final output.

The depth of the network controls how many transformations are applied. With enough hidden units, the model can represent complex relationships between input features and targets. Each layer learns to extract features that are useful for the next layer.

Training and Regularization

Training adjusts the parameters by minimizing a loss function with backpropagation. For regression, the loss is often mean squared error. For classification, cross entropy is often used. Regularization is important because large networks can overfit.

Common techniques include **dropout**, which randomly removes hidden units during training and improves generalization. **Batch normalization** normalizes the inputs to each layer, which stabilizes training and makes the network less sensitive to initialization. **Weight decay** adds a penalty on large weights to the loss function. This encourages simpler solutions and reduces overfitting.

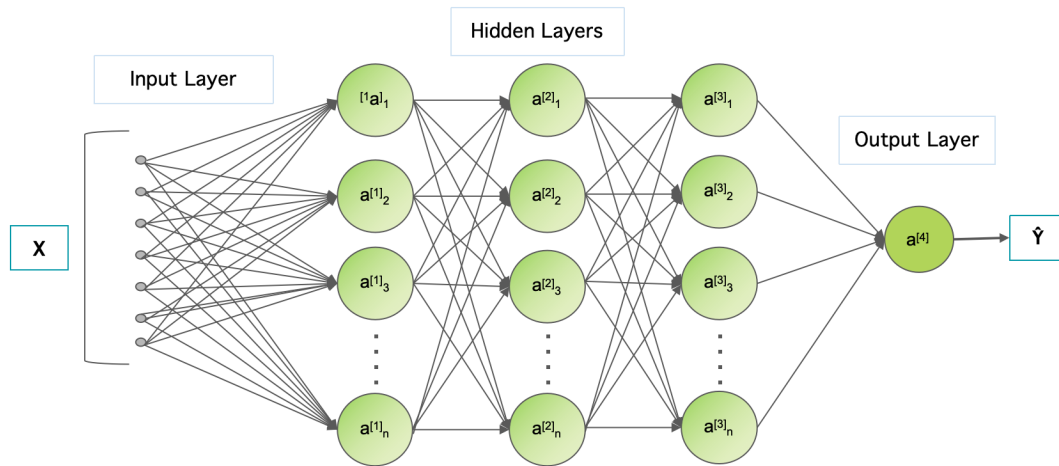


Figure 3.1: A fully connected neural network. An input vector is mapped to an output vector by stacked hidden layers.

3.2 Graph neural networks

What is a graph

A graph is a structure made of nodes and edges. Nodes represent objects, and edges represent relations between objects. Graphs can be directed or undirected. In a directed graph, an edge has a direction. In an undirected graph, the relation is symmetric.

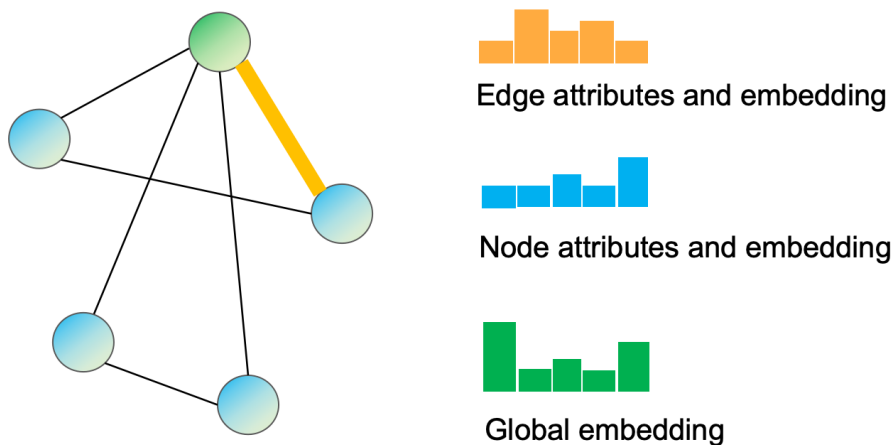


Figure 3.2: Main components of a graph. A graph can contain node attributes, edge attributes, and global information.

A graph can store information at different levels. Node attributes describe individual nodes. Edge attributes describe relations between pairs of nodes. A graph can also contain global information that describes the whole structure. These attributes are often represented as vectors and used as input to a learning model. A graph neural network is a neural network designed for graph structured data. Its goal is to learn representations that use both node features and graph structure.

Graph representation and message passing

A graph is represented by its connectivity and its node features. The connectivity is encoded by an adjacency matrix $A \in \mathbb{R}^{G \times G}$, where A_{uv} indicates whether there is an edge from node u to node v . In undirected graphs, this matrix is symmetric. In directed graphs, A_{uv} and A_{vu} may differ, which allows the representation to preserve edge direction.

The graph structure alone is not sufficient for learning. Each node must also carry a feature vector. If the graph has G nodes and each node has F input features, the node features are written as a matrix $H^{(0)} \in \mathbb{R}^{G \times F}$. Each row corresponds to one node, while the adjacency matrix specifies which nodes can exchange information. The input to the GNN is therefore given by the pair $(A, H^{(0)})$.

Other representations, such as edge lists or adjacency lists, are often used in implementation for computational efficiency, especially in sparse graphs. However, the pair $(A, H^{(0)})$ gives the clearest mathematical description of the input and is sufficient for the formulation used in this thesis.

A graph neural network operates through message passing. At each layer, every node receives information from its neighbours and uses it to update its representation. The same update rule is applied across all nodes. This shared rule allows the model to work on graphs of different sizes. The basic update rule is:

$$h_v^{(\ell+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}^{(\ell)} \right), \quad (3.7)$$

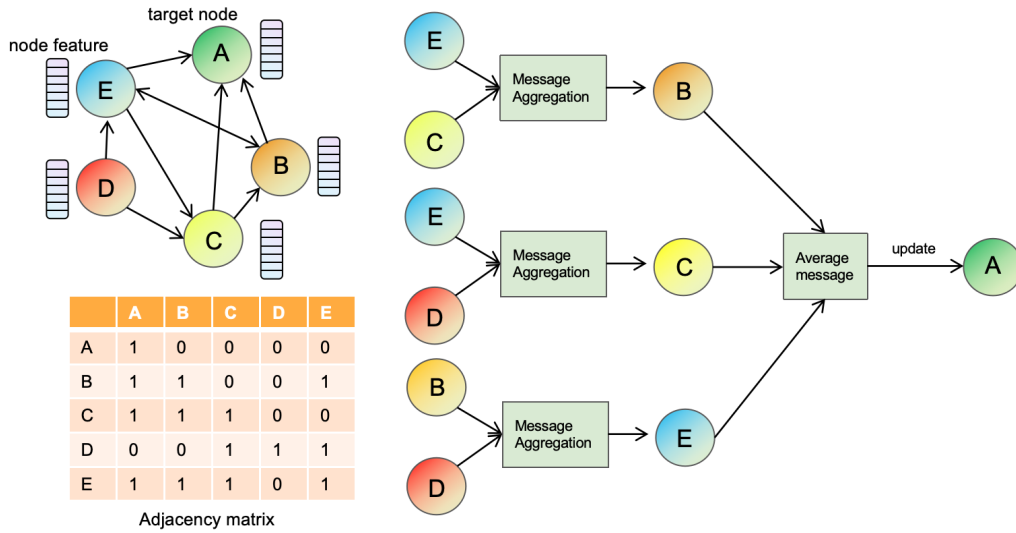


Figure 3.3: Illustration of message passing and node update in a GNN. Messages from neighbouring nodes are aggregated and then used to update the target node representation.

where $h_v^{(\ell)}$ is the state of node v at layer ℓ , $\mathcal{N}(v)$ is the set of neighbours of node v , $m_{u \rightarrow v}^{(\ell)}$ is the message from node u to node v , and σ is a nonlinear activation function.

As shown in Figure 3.3, the update of a target node has two main steps. In the first step, each neighbour sends a message based on its current feature. These messages are then aggregated, for example by a sum or an average. In the second step, the aggregated message is used to update the target node representation.

The message $m_{u \rightarrow v}^{(\ell)}$ is typically computed from the sender state $h_u^{(\ell)}$, the receiver state $h_v^{(\ell)}$, and edge features between u and v . This process happens in parallel at every node. After one layer, a node contains information from its direct neighbours. With more layers, the representation reflects a larger part of the graph.

This general message passing scheme is the common basis of many graph neural networks. Different architectures mainly differ in how messages are weighted and aggregated. In this thesis, the two most relevant variants are GCN and GAT. GCN uses a fixed normalized aggregation rule derived from the graph structure. GAT instead learns attention weights from the data, which makes the aggregation more adaptive.

Prediction

After L layers, the network produces final node representations $H^{(L)}$. These representations are not yet the final prediction. A prediction function must still be applied to them. For node level tasks, each node representation $h_v^{(L)}$ is passed to a readout function $g(\cdot)$ to produce one output for that node,

$$\hat{y}_v = g(h_v^{(L)}). \quad (3.8)$$

The full prediction is then formed by collecting the outputs from all nodes. In this thesis, each node represents one gene, so the model produces one scalar prediction for each gene and combines them into a response vector $\hat{y} \in \mathbb{R}^G$. For graph level tasks, the node representations are usually pooled into a single graph representation before prediction. This difference is important because it determines whether the model predicts per node quantities or one value for the whole graph.

Graph Convolution Network (GCN)

GCN is one of the most widely used graph neural network architectures. Its main idea is to combine node features with local graph structure in each layer. A node is updated not only from its own feature vector, but also from the features of its neighbours. This makes GCN suitable for biological networks, where the state of one gene is often related to the states of connected genes.

Let A be the adjacency matrix, let I be the identity matrix, and let $\tilde{A} = A + I$ denote the adjacency matrix after adding self loops. Let $\tilde{D}$ be the degree matrix of $\tilde{A}$. The normalized propagation matrix is

$$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} \quad (3.9)$$

A GCN layer then updates the node feature matrix $H^{(\ell)} \in \mathbb{R}^{G \times d}$ as

$$H^{(\ell+1)} = \sigma(\hat{A} H^{(\ell)} W^{(\ell)}) \quad (3.10)$$

where $W^{(\ell)} \in \mathbb{R}^{d \times d'}$ is a learnable weight matrix and $\sigma(\cdot)$ is a nonlinear activation function. This equation shows that propagation and feature transformation are performed together in each layer. The matrix $\hat{A}$ determines how information is mixed across neighbouring nodes, while $W^{(\ell)}$ learns how the aggregated signal should be transformed.

At the level of a single node, the same update can be written as

$$h_v^{(\ell+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_u \tilde{d}_v}} W^{(\ell)} h_u^{(\ell)} \right) \quad (3.11)$$

where $\tilde{d}_v = |\mathcal{N}(v)| + 1$ is the degree including the self loop. This form makes the role of normalization clearer. Each neighbour contributes to the update, but the contribution is scaled by node degrees. As a result, nodes with many neighbours do not dominate the aggregation, feature magnitudes remain more stable across layers, and each node keeps access to its own information.

Multi-layer propagation in GCN

When several GCN layers are stacked, the receptive field grows with depth. A two layer GCN can be written as

$$\begin{aligned} H^{(1)} &= \text{ReLU}(\hat{A} H^{(0)} W^{(0)}) \\ H^{(2)} &= \hat{A} H^{(1)} W^{(1)} \end{aligned} \quad (3.12)$$

The first layer aggregates information from direct neighbours, while the second layer extends the representation to two hop neighbours. The resulting node features therefore combine gene specific measurements with local network context. In this way, GCN does not treat a gene as an isolated variable. It predicts each gene under the influence of its surrounding subnetwork. This property is useful in perturbation modelling, since drug effects often spread through connected pathways rather than remaining at a single target node.

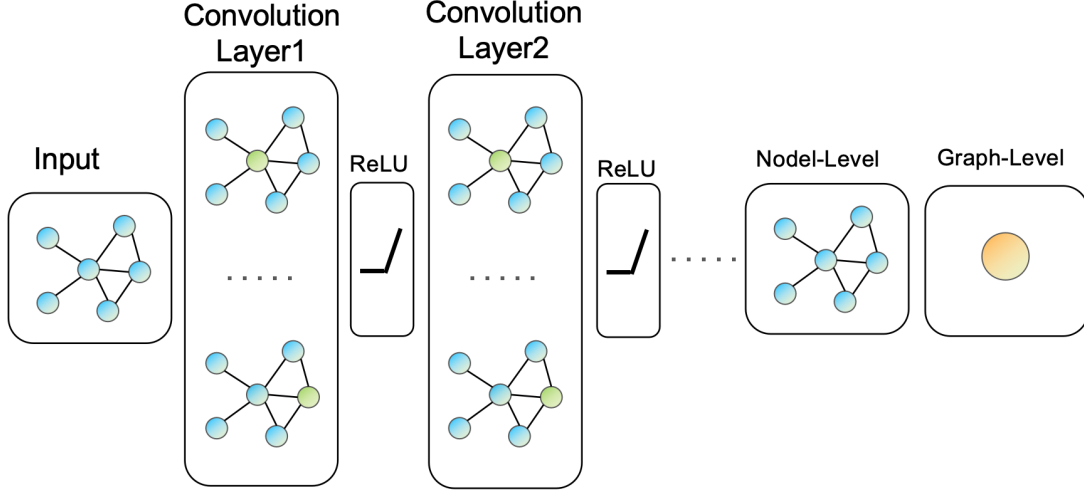


Figure 3.4: Illustration of multi layer GCN propagation. The input graph is processed by consecutive graph convolution layers, and the learned representations can be used for node level or graph level prediction.

Graph Attention Network (GAT)

GAT is a graph neural network that gives different weights to different neighbours. Instead of treating all neighbours in the same way, it learns which neighbours are more important for the update of a target node. As shown in Figure 3.5, the node features are first linearly projected. For a target node v and one of its neighbours u , the projected features are concatenated and used to compute an attention score

$$e_{u \rightarrow v} = \text{LeakyReLU} \left(a^\top \left[W h_u^{(\ell)} \parallel W h_v^{(\ell)} \right] \right), \quad (3.13)$$

where W and a are learned parameters, and $\parallel$ denotes vector concatenation. These scores are then passed through a softmax over the neighbours of v ,

$$\alpha_{u \rightarrow v} = \text{softmax}_u(e_{u \rightarrow v}), \quad (3.14)$$

so that the attention weights are positive and sum to one. The new state of node v is obtained by a weighted sum of neighbour features,

$$h_v^{(\ell+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{u \rightarrow v} W^{(\ell)} h_u^{(\ell)} \right). \quad (3.15)$$

Neighbours with larger attention weights contribute more strongly to the update. Compared with GCN, this aggregation is not fixed by the graph structure alone. It is learned from the data. GAT can also use several attention heads in parallel. Each head k produces its own intermediate representation $h_v^{(\ell+1),k}$. For intermediate layers, these outputs are often concatenated,

$$h_v^{(\ell+1)} = \parallel_{k=1}^K h_v^{(\ell+1),k}. \quad (3.16)$$

For the last layer, averaging is often used,

$$h_v^{(\ell+1)} = \frac{1}{K} \sum_{k=1}^K h_v^{(\ell+1),k}. \quad (3.17)$$

Stacking several GAT layers lets information move across a larger part of the graph. In this way, each node representation can use both local structure and learned neighbour importance.

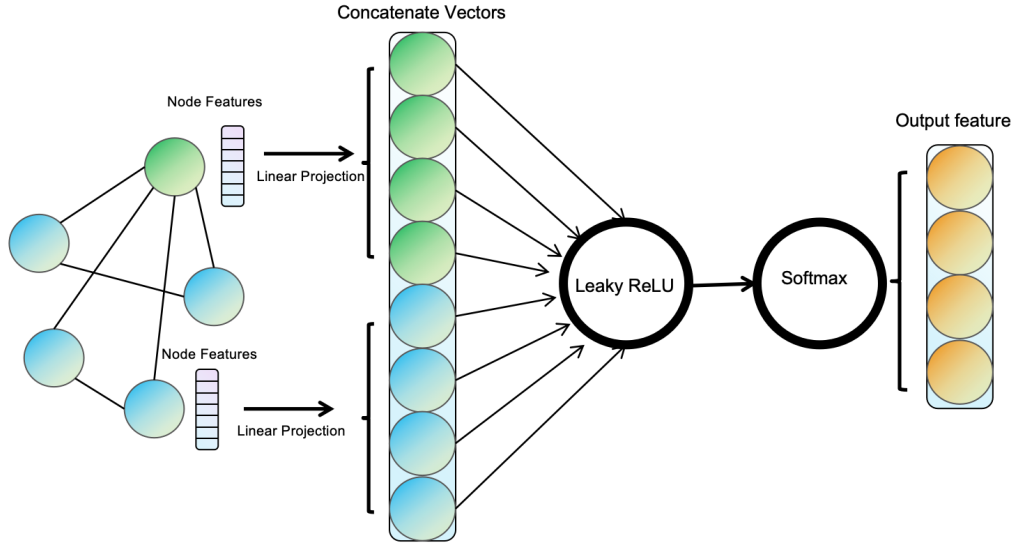


Figure 3.5: Illustration of the attention mechanism in GAT. Node features are linearly projected, concatenated, and transformed into attention weights through LeakyReLU and softmax.

3.3 Baseline model

As a baseline, a DeepCOP style fully connected network is used [16]. It does not use a gene interaction graph. The model learns a direct mapping from input features to the response vector. This baseline is included to show what can be achieved without **any network structure**.

Inputs

The input consists of three parts. The control expression vector $x_{\text{ctl}} \in \mathbb{R}^G$ describes the baseline state of the cell. The drug target vector $t \in \mathbb{R}^G$ is sparse and aligned to the same gene order as x_{ctl} . It indicates which genes are known targets of the compound. The cell identity label is encoded as a learnable embedding vector. All inputs are concatenated into one feature vector for the multilayer perceptron.

Network architecture

The concatenated input is normalized with batch normalization. It is then passed through two fully connected blocks. Each block uses a linear layer, batch normalization, ReLU activation, and dropout. This matches the two hidden layer MLP structure used in DeepCOP [16]. A final linear layer maps the hidden representation to an output vector of length G .

Output

The baseline predicts a response vector $\hat{y} \in \mathbb{R}^G$. Each entry of $\hat{y}$ corresponds to one gene in the selected gene set, and it represents the predicted change in expression after treatment relative to the matched control.

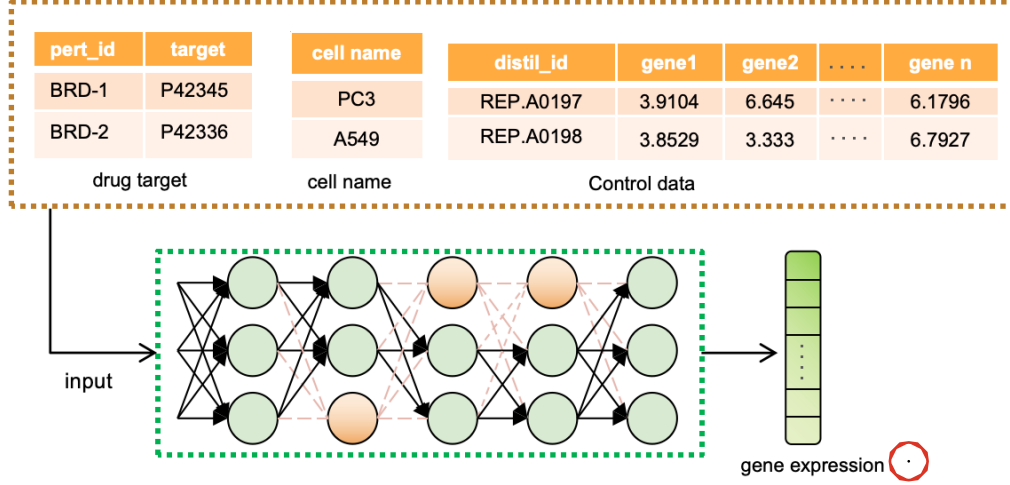


Figure 3.6: Structure of the baseline model. Drug target information, cell identity, and control gene expression are used as input to a feed forward network that predicts the gene expression response.

3.4 Proposed model

The proposed model is a graph attention network that predicts the gene expression response for each gene node. It uses a biological interaction network as the graph structure and conditions the prediction on the control profile, drug targets, and cell identity. GAT is used instead of GCN because it can learn different weights for different neighbours. This is useful when prior interaction networks are incomplete or noisy. Attention weights can also be inspected after training, which supports interpretation of the learned propagation pattern.

Graph construction

Prior biological knowledge is represented as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ built from OmniPath. Nodes correspond to genes in the selected gene set. Edges represent known interactions between genes, for example where one gene can regulate another. **If possible**, interaction type is encoded as a signed edge weight $w_{uv} \in \{+1, -1\}$. The final graph is stored as an adjacency matrix $A \in \mathbb{R}^{G \times G}$, where A_{uv} is nonzero when there is an edge from u to v . **In the sparse implementation, self loop edges are added for all nodes with weight 1 to preserve the node's own information in each layer.**

Model inputs and node features

The proposed model uses one node per gene. The starting point is the control expression profile, where each gene contributes one scalar value. For a sample with G genes, the initial node feature matrix is written as

$$H^{(0)} = \phi(x_{\text{ctl}}) \in \mathbb{R}^{G \times F}, \quad (3.18)$$

where $\phi(\cdot)$ maps each gene expression value to an F dimensional hidden representation. In the implementation, each control expression value is first projected to a hidden vector. Drug targets are represented as a binary vector aligned to the same gene order. This target vector is projected to

the same hidden space and added to the expression embedding, so target genes receive an additional input signal at the node level. Cell identity is represented by a learnable embedding. This embedding is broadcast to all nodes in the same sample and added to the initial node representation.

GAT backbone

The model uses L graph attention layers to propagate messages over $\mathcal{G}$. The number of layers L is a hyperparameter. Attention weights determine how strongly each neighbour contributes to the update, following the mechanism described in the previous section. Signed edge weights w_{uv} are used when available to encode activating or inhibiting relations. During message passing, the attention weight determines the strength of the contribution, while the edge sign determines whether the contribution is positive or negative.

Each GAT layer is implemented as a residual block. The attention output is first passed through dropout and then added to the layer input, followed by layer normalization. A position wise feed forward network is then applied to each node, followed by dropout, a second residual connection, and a second layer normalization. This design increases model capacity while keeping the graph structure fixed, and it helps stabilize training.

Prediction head

After L layers, each gene has a learned representation $h_v^{(L)}$. A single linear readout is applied to each node to produce one scalar prediction,

$$\hat{y}_v = g\left(h_v^{(L)}\right), \quad (3.19)$$

and the full response vector is $\hat{y} = [\hat{y}_1, \dots, \hat{y}_G] \in \mathbb{R}^G$. This design matches the task setting because the target is defined at the gene level. It also avoids pooling all node features into a single graph level vector, which would lose gene specific information. A deeper prediction head would add many parameters because the output must be produced for every node. Using a single output layer is enough to keep the model simple while predicting each gene separately.

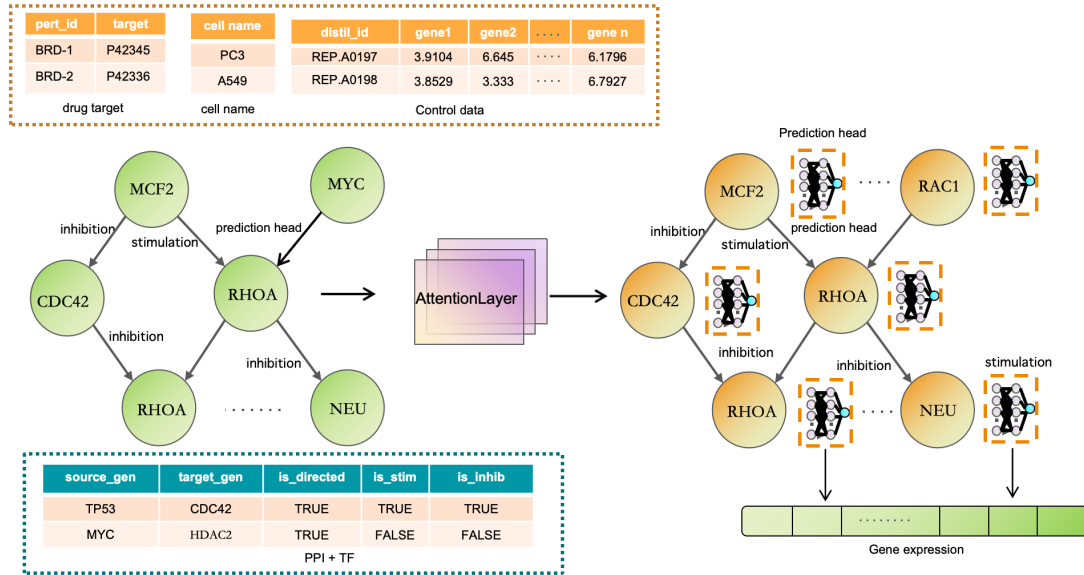


Figure 3.7: Structure of the proposed model. The model combines the biological interaction graph, control expression profile, drug target information, and cell identity to produce gene wise predictions through graph attention layers and node level prediction heads.

3.5 Training

Loss function

Training minimizes a regression loss between y and $\hat{y}$. A binary gene mask $m \in \{0, 1\}^G$ is used so that only selected genes contribute to the objective. The masked mean squared error is

$$\mathcal{L}_{\text{mse}} = \frac{1}{B \sum_g m_g} \sum_{i=1}^B \sum_{g=1}^G m_g (y_{ig} - \hat{y}_{ig})^2, \quad (3.20)$$

where B is the batch size. The objective also includes a sample level Pearson correlation coefficient (PCC) term. For each sample i , PCC is computed on the target gene set and averaged over the batch. This term focuses on the overall response pattern across genes and is less sensitive to differences in scale. The correlation loss is

$$\mathcal{L}_{\text{pcc}} = 1 - \frac{1}{B} \sum_{i=1}^B \text{corr}(y_i, \hat{y}_i). \quad (3.21)$$

Here $\text{corr}(y_i, \hat{y}_i)$ is the Pearson correlation coefficient,

$$\text{corr}(y_i, \hat{y}_i) = \frac{\sum_{g \in \mathcal{G}_{\text{tgt}}} (y_{ig} - \bar{y}_i) (\hat{y}_{ig} - \bar{\hat{y}}_i)}{\sqrt{\sum_{g \in \mathcal{G}_{\text{tgt}}} (y_{ig} - \bar{y}_i)^2} \sqrt{\sum_{g \in \mathcal{G}_{\text{tgt}}} (\hat{y}_{ig} - \bar{\hat{y}}_i)^2}}, \quad (3.22)$$

where $\bar{y}_i$ and $\bar{\hat{y}}_i$ are means over genes in $\mathcal{G}_{\text{tgt}}$.

The full prediction loss is

$$\mathcal{L}_{\text{full}} = \mathcal{L}_{\text{mse}} + \lambda_{\text{pcc}} \mathcal{L}_{\text{pcc}}, \quad (3.23)$$

This weight balances the two goals. The MSE term focuses on absolute prediction error for each gene. A larger λ_{pcc} makes the PCC term more important during training.

Counterfactual drug loss

To encourage the model to use drug target information, each training step uses two forward passes. The first pass uses the true target vector and produces $\hat{y}$ and the factual loss $\mathcal{L}_{\text{full}}$. The second pass sets the target vector to zero and produces a counterfactual prediction $\hat{y}_{\text{cf}}$ and loss $\mathcal{L}_{\text{cf}}$. This design is related to margin based objectives and counterfactual reasoning [20, 21, 22].

A simple penalty is added when the counterfactual loss is too close to the factual loss. The model should perform worse when drug targets are removed. The penalty is

$$\mathcal{L}_{\text{hinge}} = \max(0, \delta - (\mathcal{L}_{\text{cf}} - \mathcal{L}_{\text{full}})). \quad (3.24)$$

Here δ is a small gap that we want to see between the counterfactual and factual losses. If the gap is already large enough, the penalty is zero. The final loss function is

$$\mathcal{L} = \mathcal{L}_{\text{full}} + \lambda_{\text{cf}} \mathcal{L}_{\text{hinge}}. \quad (3.25)$$

This term pushes the model to make factual predictions better than counterfactual predictions. In our implementation, λ_{cf} and δ are configurable hyperparameters, with default values $\lambda_{\text{cf}} = 1.0$ and $\delta = 0.1$.

Optimization

The model is trained with the Adam optimizer and a learning rate of 5×10^{-4} . Mini batches are used, with batch size and number of epochs set as training hyperparameters. Training is monitored by reporting mean squared error and sample level PCC on the training and validation sets after each epoch.

3.6 Evaluation

Model performance is evaluated with three data split settings. Warm start uses a random split of paired examples, so the same drug and the same cell line can appear in both training and test sets. Cold drug splits by compound, so test drugs do not appear in training. Cold cell splits by cell line, so test cell lines do not appear in training. In all settings, the split is applied after treatment-control pairs have been constructed. Because one treatment profile can be paired with multiple sampled controls, repeated paired examples may share the same treatment identifier. In the cold drug and cold cell settings, these repeated pairs inherit the same drug or cell label and therefore remain on the same side of the split. This reduces leakage from repeated treatment profiles across training and test sets. By contrast, warm start is less strict and can still place closely related paired examples in both sets.

PCC is used because the task focuses on gene level response patterns. It measures how well the model matches the relative up and down regulation across genes within a sample. It is also robust to global scaling differences between y and $\hat{y}$. This makes it a useful complement to mean squared error.

Performance is reported using mean squared error and sample level Pearson correlation coefficient (PCC). The mean squared error is computed over samples and target genes,

$$\text{MSE} = \frac{1}{N|\mathcal{G}_{\text{tgt}}|} \sum_{i=1}^N \sum_{g \in \mathcal{G}_{\text{tgt}}} (y_{ig} - \hat{y}_{ig})^2, \quad (3.26)$$

where N is the number of test samples and $\mathcal{G}_{\text{tgt}}$ is the target gene set.

For a sample i , PCC is computed between y_i and $\hat{y}_i$,

$$\text{PCC}_i = \frac{\sum_g (y_{ig} - \bar{y}_i) (\hat{y}_{ig} - \bar{\hat{y}}_i)}{\sqrt{\sum_g (y_{ig} - \bar{y}_i)^2} \sqrt{\sum_g (\hat{y}_{ig} - \bar{\hat{y}}_i)^2}}, \quad (3.27)$$

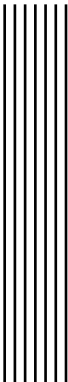
where $\bar{y}_i$ and $\bar{\hat{y}}_i$ are means over genes. The reported PCC is the average of PCC_i over samples.

Attention weights

Attention weights are extracted from the trained GAT as a method for interpretation. In each GAT layer, each edge $u \rightarrow v$ has a non negative weight $\alpha_{u \rightarrow v}$. For a fixed target node v , the weights over its incoming neighbours sum to one. This is because they are computed by applying a softmax over the neighbours of v . For each edge $u \rightarrow v$, the model first computes an unnormalized score $e_{u \rightarrow v}$ from the current node representations. Attention weights are then

$$\alpha_{u \rightarrow v} = \frac{\exp(e_{u \rightarrow v})}{\sum_{k \in \mathcal{N}(v)} \exp(e_{k \rightarrow v})}, \quad (3.28)$$

where $\mathcal{N}(v)$ is the set of incoming neighbours of node v . This normalization makes attention values comparable within the same neighbour set. The model can use multiple attention heads. Each head produces its own weights. In our analysis, head specific weights are averaged to obtain one attention value per edge. This averaging is used only in the interpretation stage. It does not change the prediction made by the model. The purpose is to obtain one edge level score that is easier to compare across samples and more stable than using one head alone. This summary may hide some head specific patterns, but it gives a more robust signal for biological interpretation. Attention is computed for each layer, so each edge has one attention value per layer. Attention weights are exported by running a forward pass. The exported file stores the directed edges as `edge_index` and the corresponding mean attention values as `attention_edge_mean`. The mean is computed by averaging over attention heads and over a set of evaluation samples. The same aggregation can also be applied within a drug group or a cell line group.



Bibliography

- [1] A. Subramanian, R. Narayan, and et al., “A gene expression signature-based approach to identify novel mechanisms of action for drugs and chemical compounds,” *Nature Biotechnology*, vol. 35, pp. 463–472, 2017.
- [2] Congressional Budget Office, “Research and development in the pharmaceutical industry,” Tech. Rep. 57025, Congressional Budget Office, Apr. 2021.
- [3] D. Brown, R. Dickman, and A. Schlessinger, “Rethinking target-based drug discovery: Challenges, innovations and the next s-curve,” *Technology Networks*, Dec. 2025. Accessed: 2026-03-14.
- [4] Annual Reviews, “Network-based approaches for drug target identification,” *Annual Review of Biomedical Data Science*, vol. 8, 2025. Discusses the challenges in drug target identification and the complexity of human diseases.
- [5] Drug Hunter, “An unexpected muscle toxicity from az’s covalent program.” <https://drughunter.com/commentary/an-unexpected-muscle-toxicity-from-az-s-covalent-program>, July 2025. Accessed: 2026-03-14.
- [6] N. C. Santos, J. Figueira-Coelho, J. Martins-Silva, and C. Saldanha, “Multidisciplinary utilization of dimethyl sulfoxide: pharmacological, cellular, and molecular aspects,” *Biochemical Pharmacology*, vol. 65, no. 7, pp. 1035–1041, 2003.
- [7] D. Türei, T. Korcsmáros, and J. Saez Rodríguez, “Omnipath: guidelines and gateway for literature curated signaling pathway resources,” *Nature Methods*, vol. 13, no. 12, pp. 966–967, 2016.
- [8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [9] S. Li, H. Hua, and S. Chen, “Graph neural networks for single-cell omics data: a review of approaches and applications,” *Briefings in Bioinformatics*, vol. 26, no. 2, 2025.
- [10] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Moncecchi, “The graph neural network model,” *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [11] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017.

-
- [12] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, vol. 30, pp. 1024–1034, 2017.
 - [13] P. Roohani, Y. Wang, Y. Wang, Y. Li, X. Gao, and Y. Chen, "Gears: Gene expression analysis with regression and structure," *Nature Communications*, vol. 14, p. 1234, 2023.
 - [14] M. Shahir, J. Smith, A. Lee, and W. Zhang, "Cellograph: Graph neural networks for single-cell perturbation prediction," *Bioinformatics*, vol. 40, no. 2, pp. 456–463, 2024.
 - [15] N. J. Evans, G. B. Mills, G. Wu, X. Song, and S. McWeeney, "Graph structured neural networks for perturbation biology," *bioRxiv*, 2024.
 - [16] G. Woo *et al.*, "Deepcop: deep learning-based approach to predict gene regulating effects of small molecules," *Bioinformatics*, vol. 36, no. 2, pp. 464–471, 2020.
 - [17] C. P. Roca *et al.*, "Regulation flow analysis discovers molecular mechanisms of action from large knowledge databases," *bioRxiv*, 2025.
 - [18] Y. Wang *et al.*, "Graphgrp: A graph neural network for predicting drug response based on gene regulatory pathways," *Bioinformatics*, vol. 38, no. 12, pp. 3127–3134, 2022.
 - [19] S. Kim, S. Bae, Y. Piao, and K. Jo, "Graph convolutional network for drug response prediction using gene expression data," *Mathematics*, vol. 9, p. 772, 2021.
 - [20] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality reduction by learning an invariant mapping," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1735–1742, 2006.
 - [21] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 815–823, 2015.
 - [22] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the gdpr," *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2017.